Principal Component Analysis and Maps for Visualisation

ES 335 / ES 678 Machine Learning, Sem I 2026 Sources: `sources/course-2026/lectures/L13b_unsupervised_learning_handbook.pdf` (Chapter 1 and Chapter 4, pp. 6–21 and 61–77 of the file)

What this file is really about:

$$\text{redundant features} \rightarrow \text{shadows} \rightarrow \text{the direction with the longest shadow} \rightarrow \text{eigenvectors} \rightarrow \text{wh}$$

How to use this file. The worked examples use the same eight training runs as the K-means note. Their mean is $\bar{x} = (5.125,\ 5.5)$. Part A builds everything from a projection onto a line. Part C is practice with answers.

Step-up path

You need. Dot products, variance and eigenvalues.

1. Project a point onto a direction and split it into kept and lost parts.
2. Write the variance along a direction as $u^\top S u$.
3. Find the best direction with a Lagrange multiplier, which gives an eigenvector.
4. Choose how many components to keep, and how units change the answer.
5. Go beyond straight lines with t-SNE, and learn what such maps cannot say.

You can stop when you can compute a small PCA by hand and say what a t-SNE picture does and does not show.

Contents

Part A Maths From the Ground Up

A1 Why reduce the number of features

Features are often redundant: a long run almost always uses much memory, so the two numbers largely repeat each other. Fewer coordinates let you plot data, give later models fewer inputs, save storage, and discard noise.

Many features also make distances less informative. For 500 points uniform in the cube $[0,1]^d$:

features d	2	10	100	1000
(farthest – nearest)/nearest	73.46	2.41	0.41	0.11

A2 Projecting a point onto a direction

A **unit vector** u has length 1. The **score** of x along u is the signed length of its shadow: $z = u^\top x$. The foot of the shadow is zu, and the rest, $r = x - zu$, is perpendicular to u.

Then Pythagoras gives $\|x\|^2 = z^2 + \|r\|^2$. Since $\|u\| = 1$, $z = \|x\| \cos\theta$.

> **Example.** For $x = (2, 3)$ and $u = (4, 1)/\sqrt{17}$: $z = 11/\sqrt{17} = 2.668$, $\|x\| = 3.606$, and the residual has length 2.425. Check: $3.606^2 = 2.668^2 + 2.425^2$.

A3 The covariance matrix

Centre the data, $\tilde{x}_i = x_i - \bar{x}$. The scores $z_i = u^\top \tilde{x}_i$ have mean 0, and variance

$$s^2(u) = \frac{1}{n}\sum_i z_i^2 = u^\top S u, \qquad S = \frac{1}{n}\sum_i \tilde{x}_i \tilde{x}_i^\top.$$

Entry S_{jk} is the covariance of features j and k, and S_{jj} is the variance of feature j. For the eight runs,

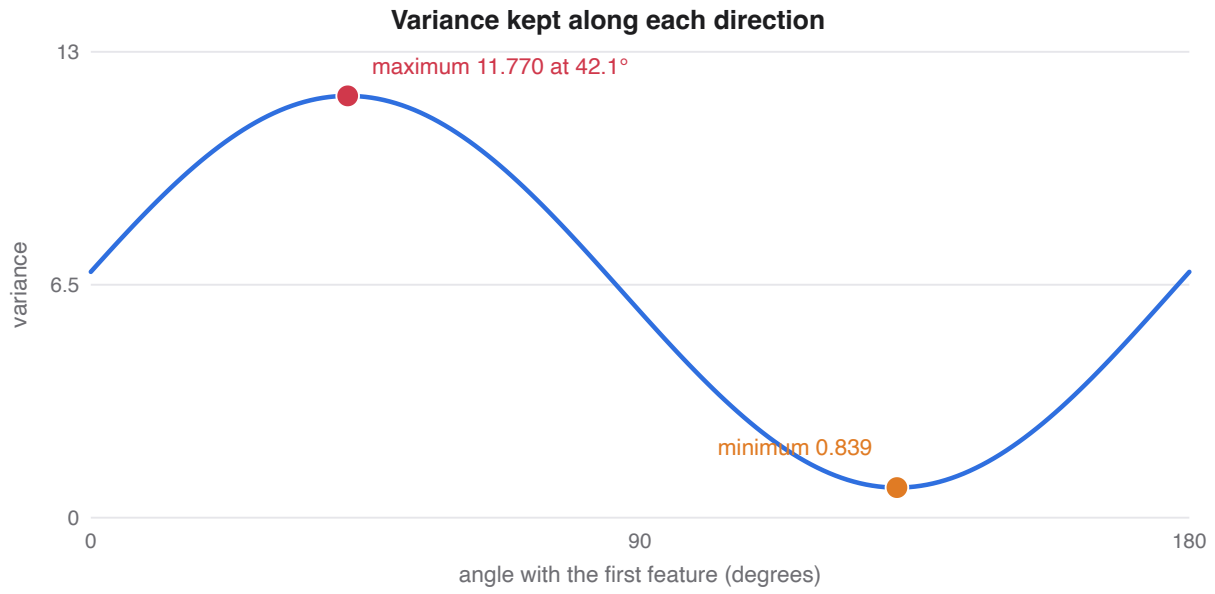
$$S = \begin{bmatrix} 6.8594 & 5.4375 \\ 5.4375 & 5.7500 \end{bmatrix}.$$

Since $u^\top S u$ is a variance it is never negative, so S is **positive semidefinite**.

A4 Variance depends on the direction

angle with x_1	unit direction	variance kept $u^\top S u$
$0°$	$(1, 0)$	6.859
$30°$	$(0.866, 0.500)$	11.291
$42.1°$	$(0.742, 0.670)$	**11.770**
$60°$	$(0.500, 0.866)$	10.736
$90°$	$(0, 1)$	5.750
$132.1°$	$(-0.670, 0.742)$	**0.839**

A diagonal keeps more than either axis because the two features vary together.



The variance kept as the direction turns. The maximum and minimum are the two eigenvalues of S, reached at perpendicular directions.

A5 Eigenvalues and eigenvectors of a symmetric matrix

A nonzero v is an **eigenvector** of S with **eigenvalue** λ if $Sv = \lambda v$. The eigenvalues solve $\det(S - \lambda I) = 0$, which for a 2×2 matrix is

$$\lambda^2 - (\operatorname{tr} S)\lambda + \det S = 0.$$

For the eight runs, $\operatorname{tr} S = 12.6094$ and $\det S = 9.8750$, so $\lambda_1 = 11.7704$ and $\lambda_2 = 0.8390$.

The **spectral theorem**: a symmetric matrix has d perpendicular unit eigenvectors, and $S = \sum_j \lambda_j v_j v_j^\top$. Two consequences follow. Each eigenvalue is the variance along its own eigenvector. The eigenvalues add up to the trace, the total variance.

A6 The first principal direction

We want the unit vector maximising $u^\top S u$. Introduce a Lagrange multiplier for $u^\top u = 1$:

$$\mathcal{L} = u^\top S u - \lambda(u^\top u - 1), \qquad \nabla_u \mathcal{L} = 2Su - 2\lambda u = 0.$$

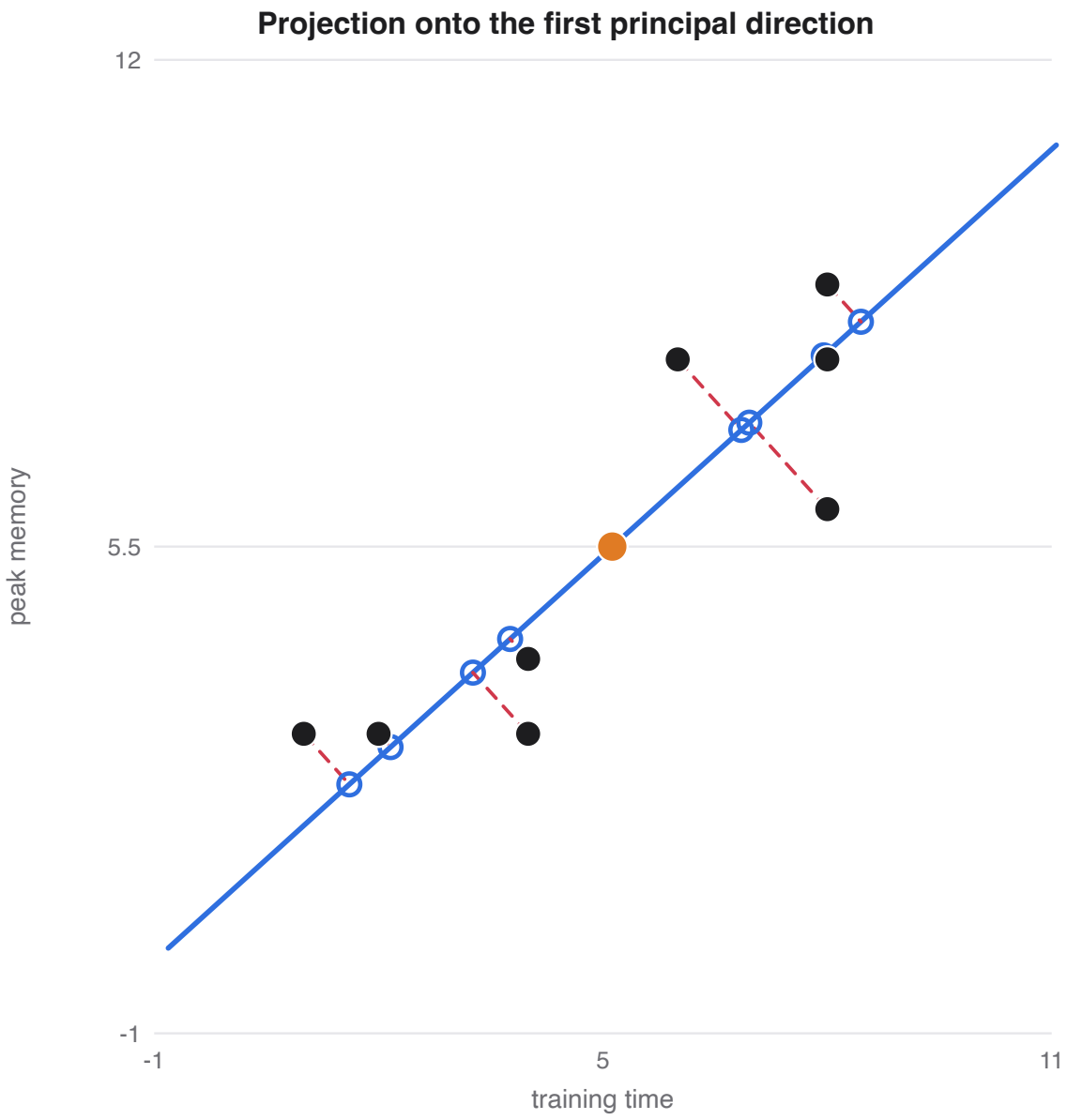
So a maximiser must satisfy $Su = \lambda u$: it is an eigenvector, and the value there is $u^\top S u = \lambda$. The maximum is the largest eigenvalue, reached at the **first principal direction** v_1. For the eight runs $v_1 = (0.7421,\ 0.6703)$, at $42.1°$.

A7 Scores and reconstruction

The **scores** are $z_{ij} = v_j^\top (x_i - \bar{x})$. Keeping only the first, run i is **reconstructed** as $\hat{x}_i = \bar{x} + z_{i1} v_1$. For run 1, the deviation is $(-1.125, -1.5)$ and $z_{11} = -1.840$.

Because v_1, v_2 are an orthonormal basis, $\tilde{x}_i = z_{i1} v_1 + z_{i2} v_2$, so the squared reconstruction error of run i is exactly z_{i2}^2.

> **Key fact.** For every direction, retained plus lost equals the fixed total, so **maximum variance is the same as minimum reconstruction error**. For the eight runs, $94.163 + 6.712 = 100.875$, which is also J for a single cluster.



The eight runs (filled), their mean, the first principal line, and each run's projection (open). The dashed segments are the reconstruction errors, and their squares add up to 6.712.

A8 Several components

Theorem. The maximum of $\sum_{j \le k} u_j^\top S u_j$ over orthonormal $u_1, \dots, u_k$ is $\lambda_1 + \cdots + \lambda_k$, reached by the top k eigenvectors.

The smallest possible reconstruction error is $n(\lambda_{k+1} + \cdots + \lambda_d)$, and the **proportion of variance explained** is $(\lambda_1 + \cdots + \lambda_k) / \sum_j \lambda_j$. Different principal scores are uncorrelated, since $v_j^\top S v_l = \lambda_l v_j^\top v_l = 0$.

A9 The algorithm, and the SVD route

1. Subtract the mean of each column.
2. If units differ, divide each column by its standard deviation.
3. Form S and find its eigen-decomposition, or take the SVD of the data matrix.
4. Scores are $Z = X_s V_k$, and reconstructions add the mean back.

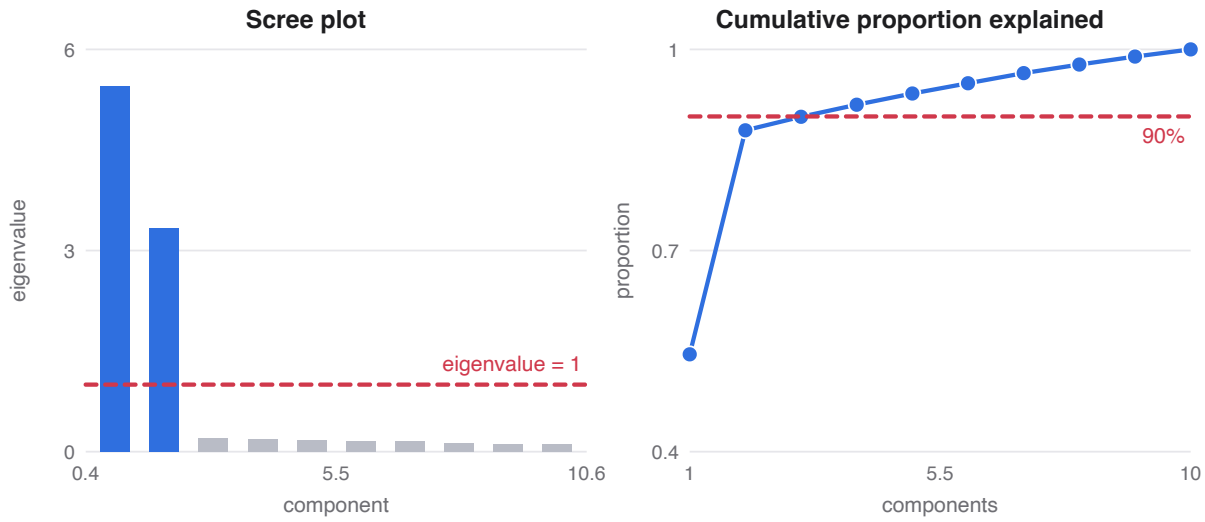
Forming S costs about nd^2. The SVD route avoids multiplying the table by itself, which squares its condition number. The SVD note covers this in detail.

A10 How many components

Three hundred runs record ten standardised metrics, with eigenvalues $5.452, 3.341, 0.202, 0.180, 0.167, 0.153, 0.151, 0.128, 0.119, 0.107$. The total is 10.

rule	result
Elbow of the scree plot	$k = 2$
Keep eigenvalues above 1	$k = 2$
Explain at least 90%	$k = 4$ (three give 89.95%)

These are conventions. The purpose decides: at most 3 for a plot, and for compression whatever error you can tolerate.



Two eigenvalues stand far above the rest, so two components explain 87.93% of the variance. The rules above disagree because each one is a convention.

A11 The effect of scaling

On the 80 runs in hours and megabytes, the first direction on raw features is the memory axis and explains 100% of the variance, simply because megabytes are larger numbers. After standardising, the two features are nearly uncorrelated ($r \approx 0.0014$), so the two directions explain 50.07% and 49.93%, and no reduction is possible.

> **Careful.** A direction of large variance need not be a direction of interest. In these data the feature that separates the two kinds of run is training time, which has the *smaller* raw variance. PCA uses no outcome, so it cannot know which variation matters.

A12 PCA on images and as a step before clustering

On 1797 images of digits, each principal direction is a pattern of pixels, and an image is rebuilt by adding patterns. Keeping $k = 21$ of 64 components keeps 90% of the variance. The mean squared error per image is 859 at $k = 2$, 315 at $k = 10$ and 116 at $k = 21$.

K-means on all 64 pixels has a purity of 79.1%, on ten components 78.7%, and on two components 59.3%. Ten components keep nearly all the structure at a fraction of the cost.

A13 Nonlinear maps: t-SNE

PCA is linear, so a rolled-up sheet gets flattened with distant layers on top of each other. **t-SNE** places points in the plane so that **neighbours remain neighbours**.

Step 1 — Similarities in the data. For each point, turn distances into probabilities:

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}.$$

σ_i is found by bisection so that the **perplexity** 2^{H_i}, an effective number of neighbours, equals a value you choose, typically 5 to 50. Then symmetrise: $p_{ij} = (p_{j|i} + p_{i|j})/2n$.

Step 2 — Similarities in the map. With a heavy-tailed Student-t kernel:

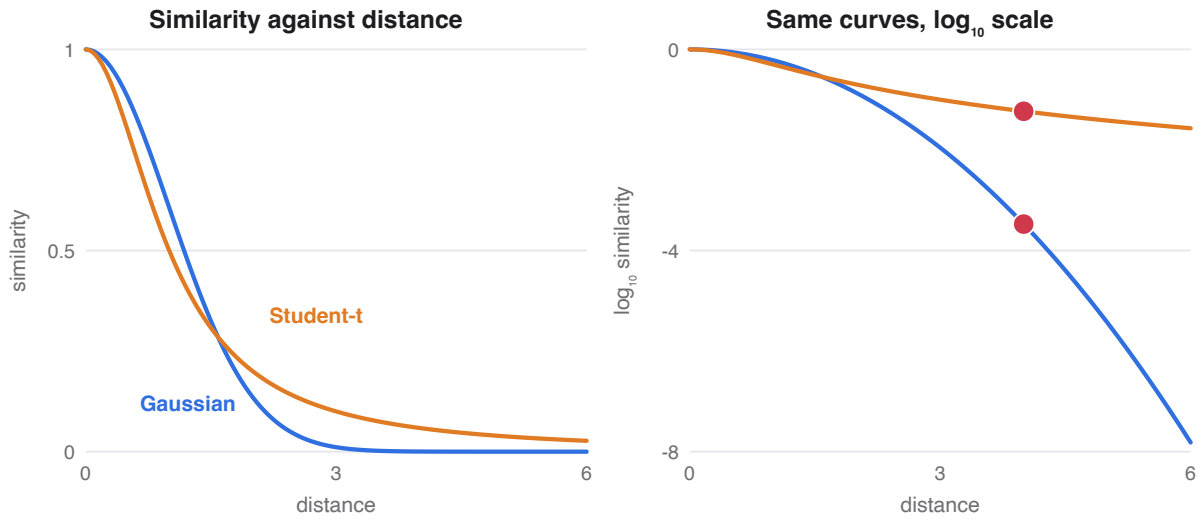
$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}.$$

Step 3 — The objective. Minimise the Kullback-Leibler divergence $C = \sum_{i \neq j} p_{ij} \log(p_{ij}/q_{ij})$, which is never negative, by Jensen's inequality.

Step 4 — The gradient.

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij}) \frac{y_i - y_j}{1 + \|y_i - y_j\|^2}.$$

Each term pulls y_i towards y_j when the pair is more similar in the data than in the map. The heavy tail relieves **crowding**, since moderately distant points can be placed far apart.



At distance 4 the Gaussian similarity is about 3×10^{-4} and the Student-t similarity about 0.06. The heavy tail is what gives moderately distant points room in the plane.

A14 UMAP, PaCMAP, and how to read such maps

UMAP links each point to its nearest neighbours and arranges the map so linked points are close, and is fast. **PaCMAP** also attracts some moderately distant pairs to keep the global arrangement. The methods differ less than their derivations suggest.

In the handbook's example, cluster B is about 7.1 times as spread as cluster A in ten dimensions, but only 1.4 times on the map. The ratio of two inter-cluster distances changes from 4.25 to 7.36.

Reading a neighbour-embedding map.

- Neighbours in the data are usually close on the map. Points close on the map need not be neighbours in the data.
- Cluster sizes and gaps between clusters on the map mean little.
- A cluster on the map is not evidence of a cluster in the data. Compare with structureless data, other seeds and other perplexities.
- Do not draw conclusions from distances or densities on the map.

A15 Unsupervised learning on learned representations

Trained models turn images and text into embedding vectors, and clustering or search then runs on those vectors, usually with cosine similarity. Vector search first compares a query with cluster centres, and only searches inside the closest clusters.

Part C Practice

P1. Find the variance kept along $u = (1, 1)/\sqrt{2}$ for the eight runs.

$u^\top S u = \tfrac{1}{2}(6.8594 + 5.75 + 2 \times 5.4375) = 11.742.$

P2. Find the first score of run 6 at $(8, 6)$.

The deviation is $(2.875, 0.5)$, so $z = 0.7421 \times 2.875 + 0.6703 \times 0.5 = 2.469.$

P3. What fraction of variance do the first two of the ten eigenvalues above explain?

$(5.4516 + 3.3414)/10 = 87.93\%.$

P4. For the eight runs, find the reconstruction error with one component.

$n\lambda_2 = 8 \times 0.839 = 6.712.$

P5. A point puts probability $1/8$ on each of 8 neighbours. What are its entropy and perplexity?

$H = 3$ bits and perplexity $2^3 = 8$.

P6. Find $\mathrm{KL}(P \| Q)$ in nats for $P = (0.5, 0.5)$ and $Q = (0.25, 0.75)$.

$0.5 \ln 2 + 0.5 \ln(2/3) = 0.3466 - 0.2027 = 0.144.$

P7. Two features are in hours and megabytes. A colleague reports that one component explains 100% of the variance. What do you say?

It is almost certainly a units effect, because megabytes are numerically huge. Standardise and run

PCA again.